

1 Introduction

This document will contain the implementation details for the design learning platform. It will be used to outline the implementation before it is developed.

2 Components

In this section, I will outline the relationship between the following components and the practical steps needed to establish the relationship:

1. Artha - Meaningful Programming Language
2. DAL - Design Abstraction Language
3. CSM - Computable Semantic Model
4. CSP - Computable Semantic Primitives
5. Engine - Semantic Reasoning Engine
6. CLP - Compressed Log Processor
7. SI - Synthesized Implementation

When discussing this framework, the core principles revolve around meaning. So I think I will first start by first establishing that meaning is first constructed by narratives that use the primitive meaningful building blocks. The narratives themselves have a structure and that structure is used to build a semantic model written in the design abstraction language. Since the meaning was built from computable semantic primitives, which are themselves established using the DAL, the semantic model becomes executable and becomes a computable semantic model.

Once meaning has been established, now it can be used in another narrative. It is a compositional system where meaning is composed through narratives and used to compose more meaning. Since the meaning is built from computable semantic primitives and it has an unambiguous structure, it can be synthesized into an implementation. In the process, the meaning of the software system has been automatically developed in a programming language.

The engine reasons about the meaning using the structure established by the computable semantic model. Since the structure is unambiguous and meaningful, it is able to reason about it to enforce its own definition of correctness and to also identify missing or incorrect meaning. Through the synthesis and instrumentation, the necessary information needed to replay the execution is preserved using domain specific compression and the engine is able to diagnose any failures and enable the meaning to expand through root cause analysis. This process automatically manages the software system and enables a deterministic learning loop.

Since the meaning of the participant is specified as part of the narrative, its domain structure can also be included in that meaning. So the meaning is what CLP uses to perform the domain specific compression, the domain structure is part of its meaning. If you imagine a story, the description of the character (which is part of the meaning) can be as detailed and unambiguous as needed, in this case, that description can be extended to include the domain structure of the participant.

Of course, there is more information than just domain structure, the meaning can also tell you how the data changes so that it can be exploited to optimally compress this. There is a difference between knowing a value is a float and that it is a GPS coordinates latitude or ambient temperature. It also makes the data itself meaningful, if its GPS coordinates, you know when there is movement and when there is no movement. Also, since the meaning is already known, it can be logged in a way that significantly improves the ingestion performance. There are lots of implications for this but I'll discuss them later in the domain specific compression section. Ultimately, I just see meaning as the elimination of ambiguity in the domain (which CLP benefits from) and this framework provides the means to fully eliminate ambiguity in the meaning.

Next, I will discuss how each component can be practically implemented.

2.1 Design Abstraction Language

The Design Abstraction Language (DAL) itself is just a structure that can be used to construct the meaning of a design. I've written a basic AST library that can be used to establish blocks and commands with arguments, this is sufficient for now. Its the structures that are built with this that define the meaning.

2.2 Computable Semantic Primitives

I think this is the best place to start because it introduces how meaning is established with the DAL while also introducing the first entries into the semantic toolkit that can be used to construct narratives. So next, I will present the CSP for add and then I will discuss each part of it.

```
1  design("add")
2  type("primitive")
3
4  description("Adds two numbers and produces a result. The numbers must have type float
   ↪ or int and the result will be float or int type.")
5
6  narrative_structure {
7    # Example: store result of add cost_of_bagel and cost_of_coffee in total_cost
8    {
9      syntax: "store result of add {addend1} and {addend2} in {resultName}"
10    }
11  }
12
13  participants {
14    participant(name="addend1", required="true")
15    participant(name="addend2", required="true")
16    participant(name="resultName", required="true")
17  }
18
19  returns {
20    participant(
21      name="result",
22      type="float" or "int",
23      description="Result of the addition"
24    )
25  }
26
27  behavior add {
28    # Get the addends from the world state
29    getFromWorldState(name="addend1", role="addend1")
30    getFromWorldState(name="addend2", role="addend2")
31    getFromWorldState(name="resultName", role="resultName")
32
33    # adds the two values and store result in the value of resultName
34    _adds(result, addend1, addend2)
35
36    # Update the world state
37    updateWorldState(
38      name=resultName,
39      value=result,
40      type="float" or "int"
41    )
42  }
43
44  run (add)
```

```

45
46
47     invariants {
48         assert({
49             id:1,
50             participants: ["addend1"],
51             property: ["type"]
52             condition: "float" || "int"
53             description: "Addend1 must be a whole number or float.",
54             invalidValues: [
55                 {
56                     value: "string_value"
57                     description: "A string is provided as an addend."
58                 }
59             ]
60         }),
61         assert({
62             id:2,
63             participants: ["addend2"],
64             property: ["type"]
65             condition: "float" || "int"
66             description: "Addend2 must be a whole number or float.",
67             invalidValues: [
68                 {
69                     value: "string_value"
70                     description: "A string is provided as an addend."
71                 }
72             ]
73         })
74     }
75
76     generate{
77         {
78             type: "success",
79             sentence: f"Added {addend1} to {addend2} and resulted in {result}.",
80         },
81         {
82             type: "fail_id_1",
83             sentence: f"Failed to add the two numbers because addend1 was not a float or
84                 ↪ int.",
85         },
86         {
87             type: "fail_id_2",
88             sentence: f"Failed to add the two numbers because addend2 was not a float or
89                 ↪ int.",
90         }
91     }

```

```

1  def add(addend1, addend2):
2      return addend1 + addend2

```
